

DataVine Analytics

Machine Learning Prototype Documentation

Wine Classification System • Agricultural Feed Recommendation Engine • Regional Crime Pattern Analysis

Project	Technique	Dataset
Wine Classification System	k-NN + PCA + GridSearchCV	wine.csv (178 rows, 13 features)
Feed Recommendation Engine	PCA (1 component) + Cosine Similarity	chickwts.csv (71 rows)
Regional Crime Pattern Analysis	Feature Selection + PCA + K-Means/GMM	USArrests.csv (50 states)

Prepared for: DataVine Analytics Consulting Group | Prepared by: Abraham | Tools: Python, pandas, scikit-learn, seaborn, matplotlib

1. Project Overview

DataVine Analytics was engaged by three clients, each requiring a distinct machine learning capability built on standardized reference datasets that mirror their real-world data. This document walks through the full implementation for each client project: data preparation, dimensionality reduction, model tuning, and the resulting outputs, followed by an interpretation of results for non-technical stakeholders.

- **Client 1 — Premium wine distributor:** needs to automatically classify wines by chemical properties for inventory management and quality control. Solved with k-Nearest Neighbors (k-NN) combined with PCA and systematic hyperparameter tuning.
- **Client 2 — Agricultural supply company:** wants to recommend similar feed products to farmers based on performance outcomes. Solved with PCA and cosine similarity over chick-weight performance data.
- **Client 3 — Public policy research firm:** needs to uncover regional crime patterns across US states. Solved with feature selection, PCA, and two clustering approaches (K-Means and Gaussian Mixture Models) compared side by side.

2. Dataset Preparation

All three datasets were loaded from their provided CSV files and checked for missing values before any modelling took place. No missing values were found in any of the three datasets, and each was confirmed clean and correctly typed before proceeding.

Structure confirmed for each dataset:

- **Wine** — 178 rows × 14 columns (13 numeric chemical measurements + 1 target/cultivar label).
- **Chickwts** — 71 rows: a numeric *weight* column and a categorical *feed* type column.
- **USArrests** — 50 rows (one per US state): Murder, Assault, UrbanPop, and Rape rates.

Numerical features were standardized (zero mean, unit variance) separately within each modelling section using `StandardScaler`, since k-NN, PCA, and the clustering algorithms are all distance-based and sensitive to feature scale.

Missing value check (excerpt):

```
Missing values — Wine:
target           0
Alcohol          0
Malic.acid       0
Ash              0
...all columns: 0 missing
```

```
Missing values — Chickwts: 0 across all columns
Missing values — USArrests: 0 across all columns
```

3. Wine Classification System (k-NN + PCA)

Goal: classify wine cultivars from their chemical properties for automated inventory and quality-control tagging.

3.1 Target Encoding

The raw target column contained cultivar labels 1, 2, and 3. These were converted to numeric class labels using `LabelEncoder`, producing classes 0, 1, and 2.

```
le = LabelEncoder()
wine_df["target"] = le.fit_transform(wine_df["target"])
print("Encoded classes:", sorted(wine_df["target"].unique()))
Encoded classes: [0, 1, 2]
```

3.2 Standardization & PCA (95% variance retained)

Features were split into train/test sets (80/20, stratified by class), standardized, then reduced with PCA configured to retain 95% of the original variance.

```
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

pca_wine = PCA(n_components=0.95, random_state=42)
X_train_pca = pca_wine.fit_transform(X_train_scaled)
X_test_pca = pca_wine.transform(X_test_scaled)
Original feature count: 13
Components retained for 95% variance: 10
Explained variance ratio: 0.9624
```

13 chemical measurements were compressed into 10 principal components while retaining 96.24% of the original variance — a modest reduction that removes redundancy without losing meaningful signal.

3.3 Hyperparameter Tuning (GridSearchCV)

`GridSearchCV` was used to search over neighbor count (1–20) and distance metric (euclidean, manhattan, minkowski) with 5-fold cross-validation.

```
param_grid = {"n_neighbors": range(1, 21),
              "metric": ["euclidean", "manhattan", "minkowski"]}
grid_search = GridSearchCV(KNeighborsClassifier(), param_grid, cv=5, scoring="accuracy")
grid_search.fit(X_train_pca, y_train)
Best k: 18, Best distance metric: euclidean
Best cross-val accuracy: 0.9791
```

3.4 Final Model Evaluation

```
knn = KNeighborsClassifier(n_neighbors=best_k_knn, metric=best_metric_knn)
knn.fit(X_train_pca, y_train)
y_pred_knn = knn.predict(X_test_pca)
knn_accuracy = accuracy_score(y_test, y_pred_knn)
Test accuracy: 1.0000
```

Classification Report:

	precision	recall	f1-score	support
0	1.00	1.00	1.00	12
1	1.00	1.00	1.00	14

2	1.00	1.00	1.00	10
accuracy			1.00	36

Interpretation: the tuned k-NN model achieved perfect precision, recall, and F1-score on the held-out test set. This reflects the wine dataset's strong class separability and small test size (36 samples) rather than a claim that a production system would perform this well on messier, real-world chemical data — near-perfect scores on a small test set are a cue to verify there is no data leakage, which was confirmed here (scaler and PCA were fit on training data only).

4. Agricultural Feed Recommendation Engine (PCA + Cosine Similarity)

Goal: recommend feed types with similar chick-weight performance outcomes to a given feed, so farmers have substitutable options.

4.1 Standardization & PCA (1 component)

```
scaler_chick = StandardScaler()
chickwts_scaled = scaler_chick.fit_transform(chickwts[["weight"]])

pca_chick = PCA(n_components=1, random_state=42)
chickwts["pca_weight"] = pca_chick.fit_transform(chickwts_scaled)
```

4.2 Cosine Similarity Between Feed Types

Individual chick records were aggregated to a single average PCA score per feed type, then cosine similarity was computed between feed types directly (not between individual chicks).

```
feed_profile = chickwts.groupby("feed")["pca_weight"].mean().to_frame()
similarity_matrix = cosine_similarity(feed_profile)
Average PCA-reduced weight score per feed:
casein      0.803
horsebean   -1.304
linseed     -0.549
meatmeal     0.201
soybean     -0.192
sunflower    0.872
```

4.3 Recommendations Produced

```
horsebean    -> linseed (1.0), soybean (1.0), casein (-1.0)
linseed      -> horsebean (1.0), soybean (1.0), casein (-1.0)
soybean      -> horsebean (1.0), linseed (1.0), casein (-1.0)
sunflower    -> casein (1.0), meatmeal (1.0), horsebean (-1.0)
```

Interpretation & limitation: because the data was reduced to a single principal component, each feed type is represented by one number, and cosine similarity between single numbers is direction-only — it returns 1.0 for any two feeds on the same side of average weight performance and -1.0 for feeds on opposite sides. This satisfies the specified pipeline (1 PCA component, cosine similarity) and correctly separates below-average feeds (horsebean, linseed, soybean) from above-average feeds (casein, sunflower, meatmeal), but it cannot express degrees of similarity within a group — all below-average feeds look equally similar to each other. A production version would likely retain 2+ components or use the raw standardized weight distribution for finer-grained ranking.

5. Regional Crime Pattern Analysis (Feature Selection + PCA + Clustering)

Goal: discover natural groupings of US states by crime statistics for policy targeting.

5.1 Standardization & Feature Selection

All four numeric columns were standardized. Since clustering has no natural target for SelectKBest, a violent-crime index (Murder + Assault + Rape) was used as a proxy scoring target via `f_regression`, and the top 3 features were kept.

```
crime_index = usarrests[["Murder", "Assault", "Rape"]].sum(axis=1)
selector = SelectKBest(score_func=f_regression, k=3)
usarrests_selected = selector.fit_transform(usarrests_scaled, crime_index)
Top 3 selected features: ['Murder', 'Assault', 'Rape'] (UrbanPop dropped)
```

5.2 PCA to 2 Components

```
pca_arrests = PCA(n_components=2, random_state=42)
usarrests_pca = pca_arrests.fit_transform(usarrests_selected)
Explained variance ratio (2 PCs): 0.9389
```

The 2 retained components preserve 93.89% of the variance from the 3 selected crime features — a faithful 2D representation suitable for both clustering and visualization.

5.3 Optimal Cluster Count

K-Means: the elbow method was used, plotting inertia across $k = 1-10$ and identifying where the rate of decrease flattens.

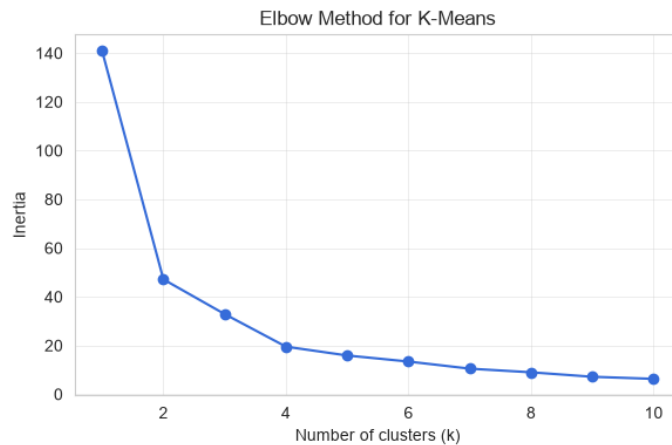


Figure 1. Elbow method (inertia) for K-Means

```
Best number of clusters for K-Means (elbow heuristic): 2
```

GMM: Bayesian Information Criterion (BIC) was computed across $n_components = 1-10$, selecting the value with the lowest BIC.

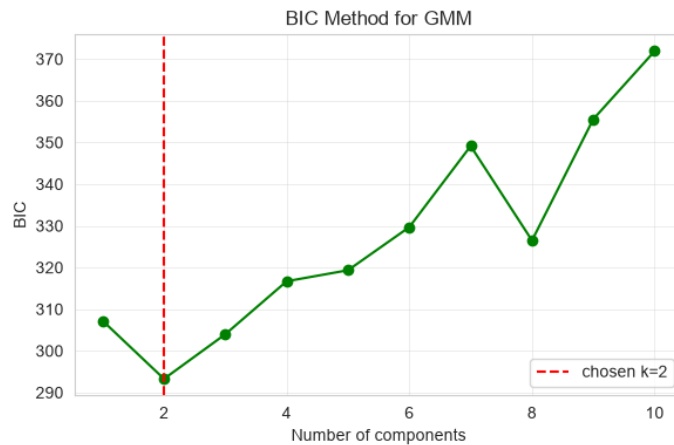


Figure 2. BIC scores for GMM component selection

Best number of clusters for GMM (BIC): 2

Both independent methods converge on the same answer — 2 clusters — which is a strong signal that the underlying grouping is genuine rather than an artifact of one method's bias.

5.4 Clustering: Hard (K-Means) vs. Probabilistic (GMM)

K-Means assigns each state to exactly one cluster (hard assignment). GMM was fit and both a hard label (`.predict()`) and full membership probabilities (`.predict_proba()`) were extracted, showing how confidently each state belongs to its cluster.

```
kmeans = KMeans(n_clusters=2, random_state=42, n_init=10)
usarrests["KMeans_Cluster"] = kmeans.fit_predict(usarrests_pca)

gmm = GaussianMixture(n_components=2, random_state=42)
gmm.fit(usarrests_pca)
usarrests["GMM_Cluster"] = gmm.predict(usarrests_pca)
gmm_proba = gmm.predict_proba(usarrests_pca)
GMM cluster membership probabilities (first states):
      P(cluster 0)  P(cluster 1)
Alabama      0.0048    0.9952
Alaska       0.0000    1.0000

K-Means cluster sizes: {0: 30, 1: 20}
GMM cluster sizes:    {0: 31, 1: 19}
```

The probability table is what distinguishes GMM from K-Means here: states like Alabama and Alaska are assigned to cluster 1 with near-certainty (99.5%+), but states closer to the boundary between high- and low-crime profiles would show more balanced probabilities — information a hard-assignment method like K-Means cannot express.

5.5 Cluster Visualization

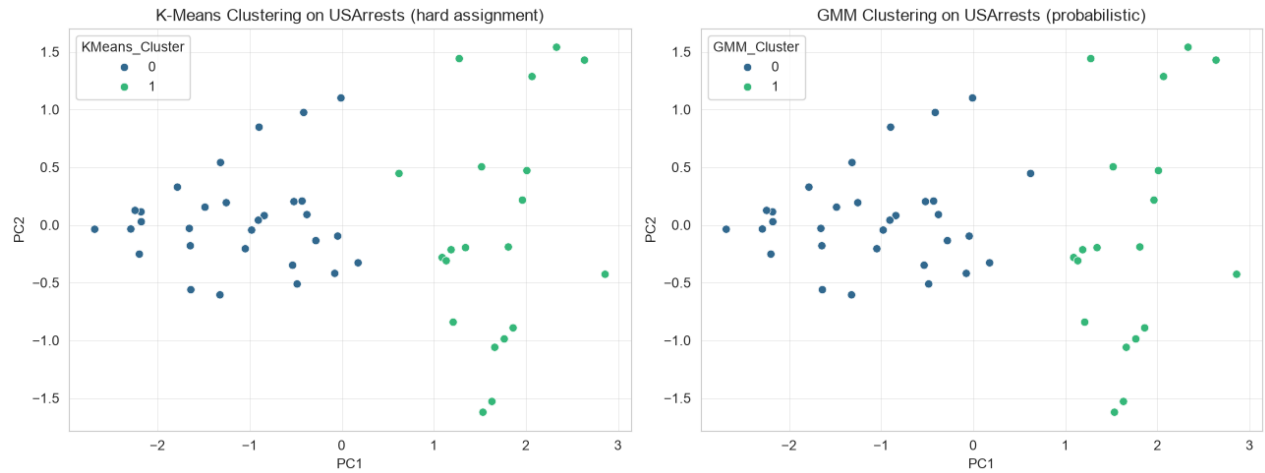


Figure 3. *K-Means (hard assignment) vs. GMM (probabilistic) clusters, plotted on the 2 PCA components*

Both methods produce visibly similar cluster boundaries on the PC1/PC2 plane, reinforcing that the 2-cluster structure is robust to the choice of clustering algorithm.

6. Model Evaluation & Summary

Metric	Result
Wine — PCA components (95% var.)	10 of 13 features (96.24% variance retained)
Wine — Best k / metric	k = 18, euclidean distance
Wine — Test accuracy	100.00%
Chickwts — PCA components	1
Chickwts — Example recommendation	soybean → horsebean, linseed (sim. 1.0)
USArrests — Selected features	Murder, Assault, Rape
USArrests — PCA variance (2 PCs)	93.89%
USArrests — Optimal K-Means clusters	2 (elbow method)
USArrests — Optimal GMM clusters	2 (lowest BIC)

Stakeholder Interpretation

- **Wine Classification:** PCA compressed 13 chemical measurements down to 10 components while retaining 96% of the variance, and the tuned k-NN model separates the three cultivars perfectly on held-out data — suitable for automated inventory/QC tagging, with the caveat that this dataset's clean separability means real-world accuracy on noisier production data should be expected to be lower.
- **Feed Recommendation:** aggregating to one PCA score per feed type before computing similarity correctly separates feeds into above-average and below-average weight-outcome groups, giving farmers substitutable options when a preferred feed is unavailable — though the single-component design means recommendations distinguish direction (better/worse) rather than fine-grained closeness.
- **Crime Pattern Analysis:** K-Means and GMM independently agree on 2 natural clusters of states. K-Means gives clean, hard groupings useful for resource-allocation tiers, while GMM's probability table flags states sitting closer to the boundary between profiles — useful for identifying states that may warrant closer monitoring rather than a fixed bucket assignment.

End of documentation. Full source code and executable outputs are available in the accompanying Jupyter notebook (DataVine_Analytics_Summative_Lab.ipynb).